# Reset Early, Reset Often, Eliminate Models

## How aggressively resetting connection tableaux can saturate the SAT solver and find an unsatisfiability proof faster.

Martin S. Fixman[1], Fredrik Rømming[1], Sean B. Holden[1]

[1]  UNIVERSITY OF CAMBRIDGE

Vampire Workshop, FLoC 2026

# Knights and Knaves



A traveller meets a husband and wife. He knows that either both of them are *knights*, who always tell the truth, or *knaves*, who always lie.

The husband makes a statement, and the traveller is given three clues.

1. If the husband's statement is false, then his wife is a knight.
2. If the wife is a knight, then the husband's statement is true.
3. If the husband is a knight, then his wife is a knight.

Smullyan (1978): *What Is the Name of This Book?: The Riddle of Dracula and Other Logical Puzzles*. Englewood Cliffs, NJ: Prentice-Hall

PUZ020-1.p

$$a(\eta) \qquad \neg b(\eta)$$

$$\neg a(A) \vee \neg c\big(\phi(A)\big) \vee b(A)$$

$$c\big(\phi(\eta)\big) \vee b(\gamma) \qquad \neg b(\gamma) \vee c\big(\phi(\eta)\big)$$

$a(A)$ $A$ is a person.

$b(A)$ $A$ is a knight.

$c(B)$ $B$ is a true statement.

$d(A)$ $A$ is a knave.

$e(A, B)$ $A$ says statement $B$.

$\phi(A)$ statement $\phi$ by $A$.

$\eta$ the husband.

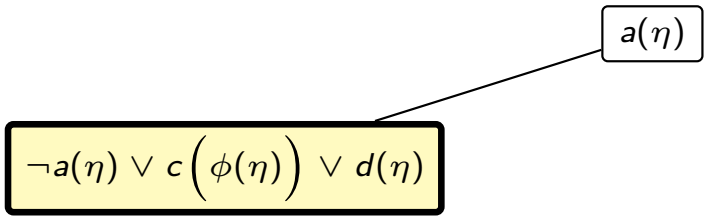
$\gamma$ the wife.

Smullyan (1978): *What Is the Name of This Book?: The Riddle of Dracula and Other Logical Puzzles*. Englewood Cliffs, NJ: Prentice-Hall
Sutcliffe (2017): *The TPTP Problem Library and Associated Infrastructure: From CNF to TH0, TPTP v6.4.0*. *Journal of Automated Reasoning* 59(4)

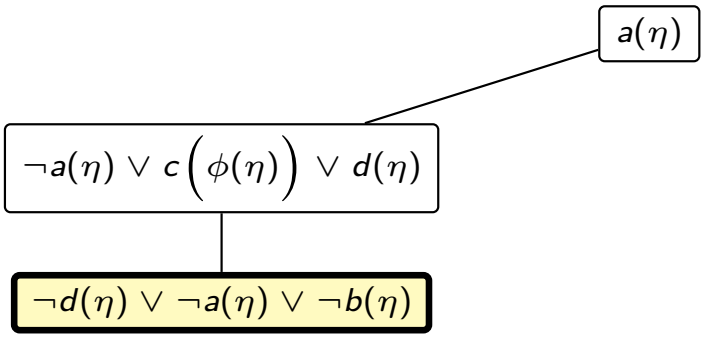
# leanTAP

$a(\eta)$

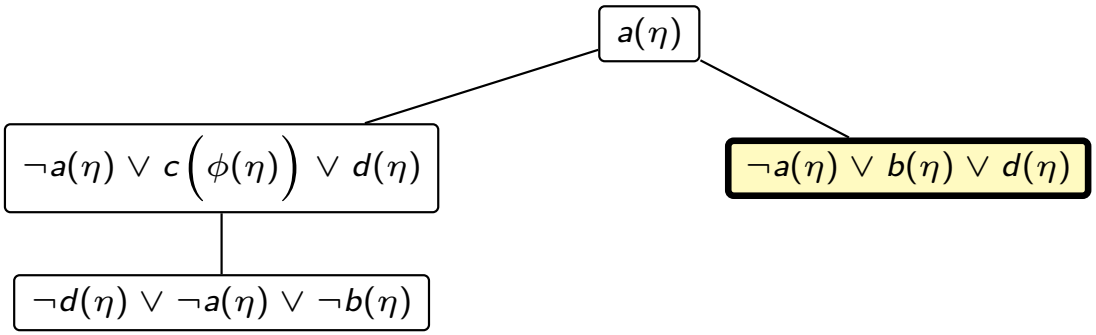
# leanTAP



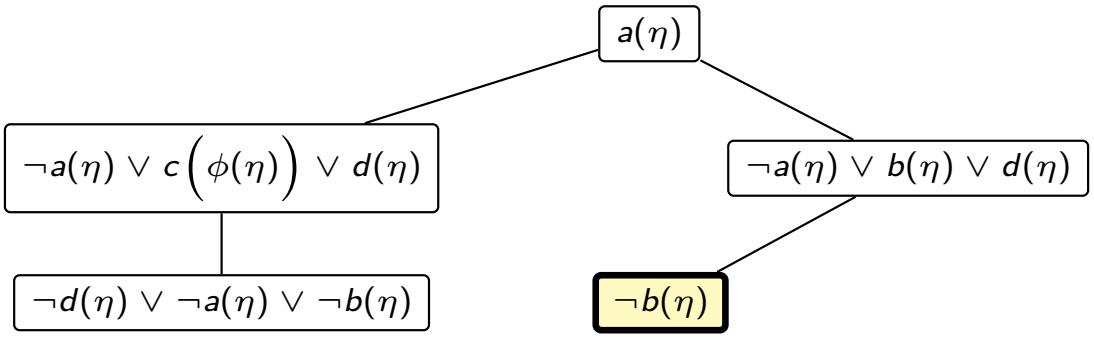
# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$

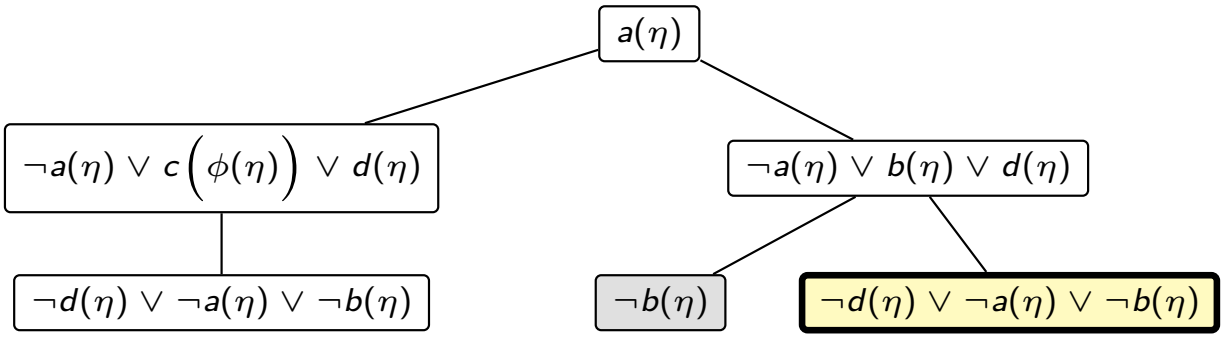
# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee b(\eta) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$

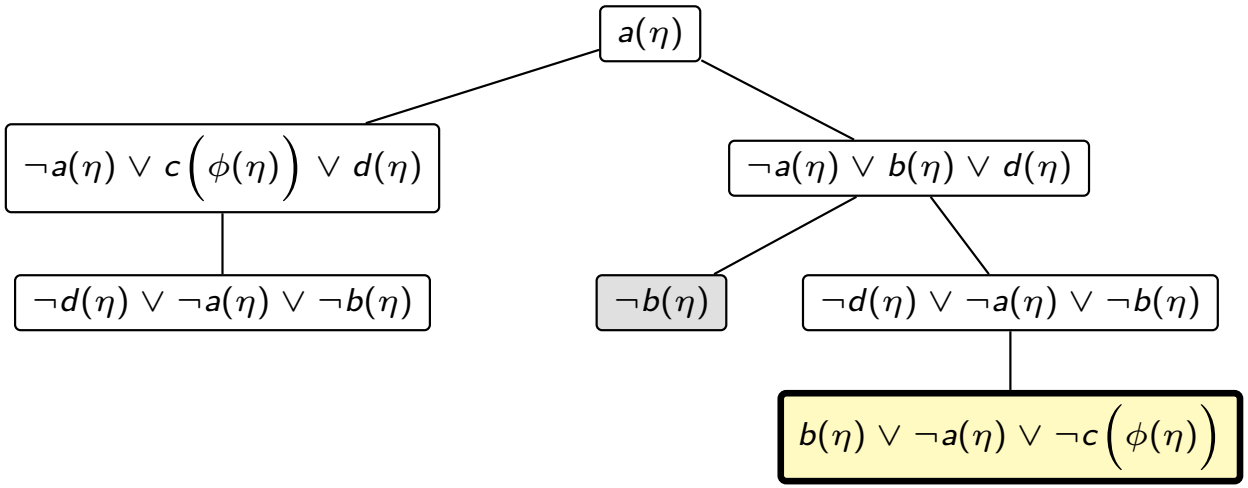
# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee b(\eta) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$\neg b(\eta)$

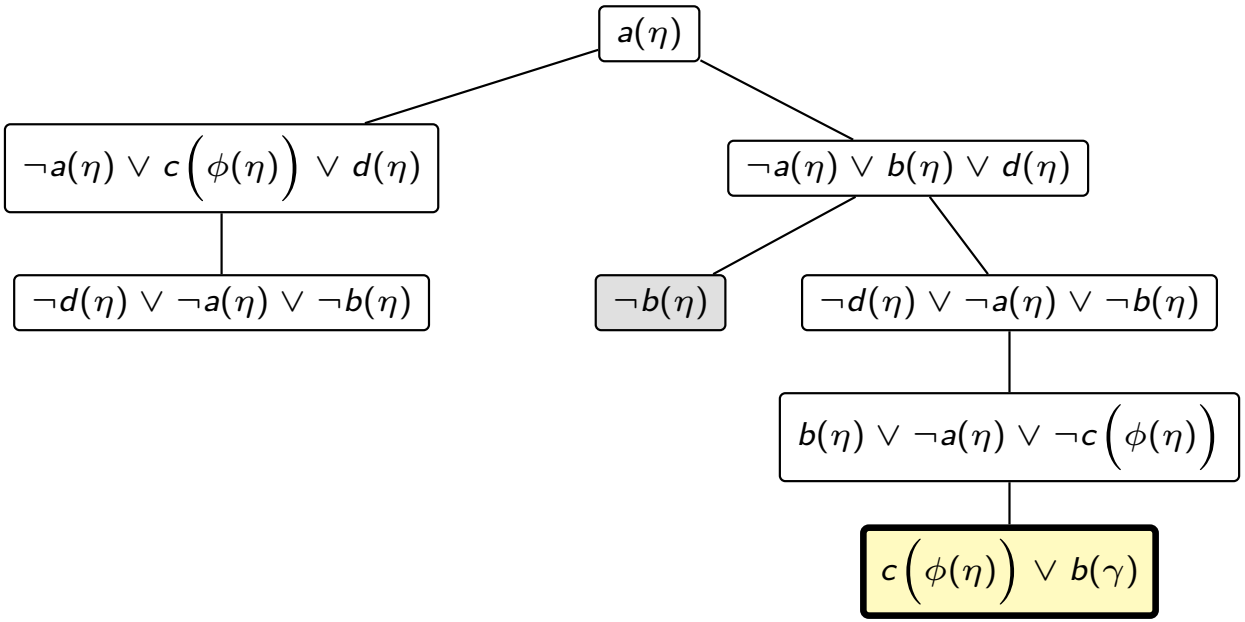
# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee b(\eta) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$\neg b(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$

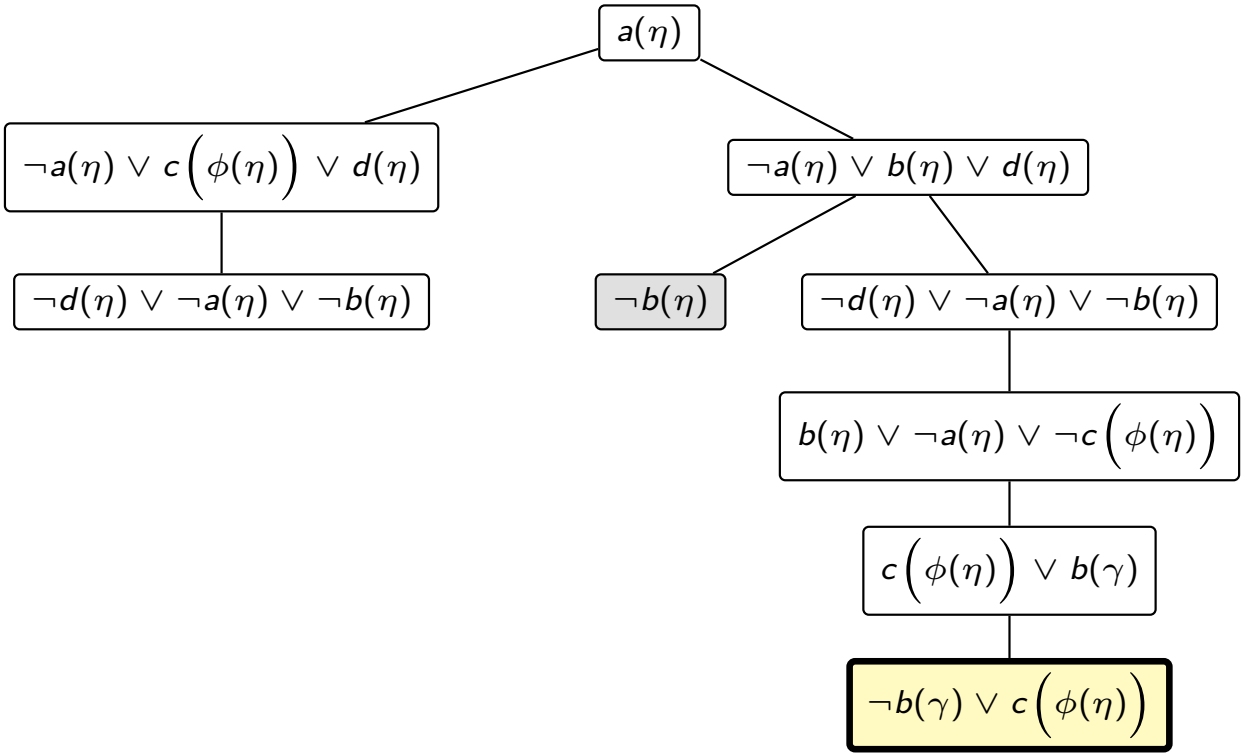
# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee b(\eta) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$\neg b(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$b(\eta) \vee \neg a(\eta) \vee \neg c\left(\phi(\eta)\right)$

# leanTAP


$a(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee b(\eta) \vee d(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$\neg b(\eta)$
$\neg d(\eta) \vee \neg a(\eta) \vee \neg b(\eta)$
$b(\eta) \vee \neg a(\eta) \vee \neg c\left(\phi(\eta)\right)$
$c\left(\phi(\eta)\right) \vee b(\gamma)$

# leanTAP



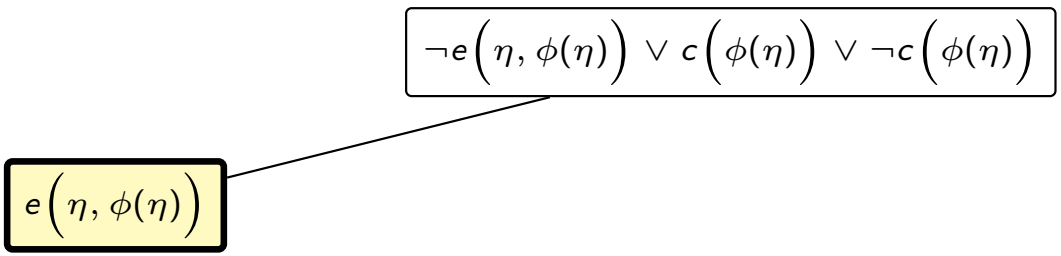

total steps: 2,563

# leanCoP

$$\neg e\Big(\eta, \phi(\eta)\Big) \vee c\Big(\phi(\eta)\Big) \vee \neg c\Big(\phi(\eta)\Big)$$

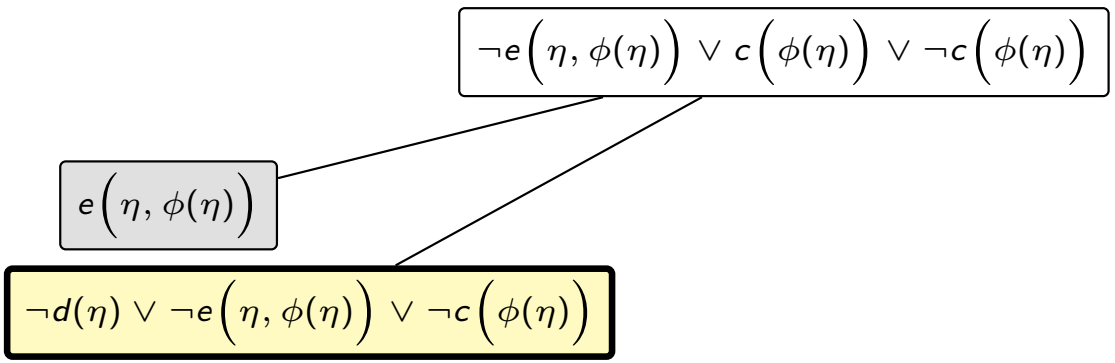

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



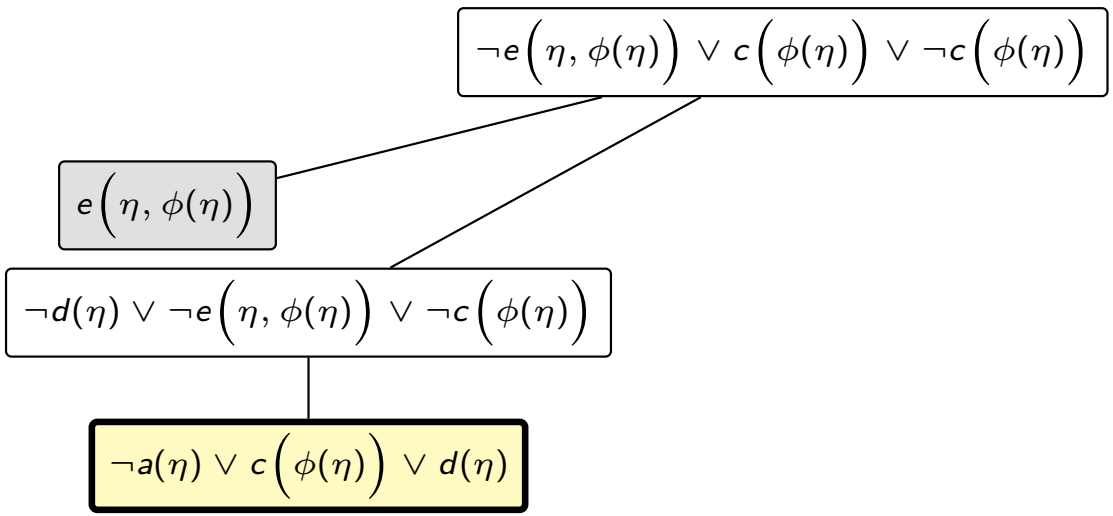

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



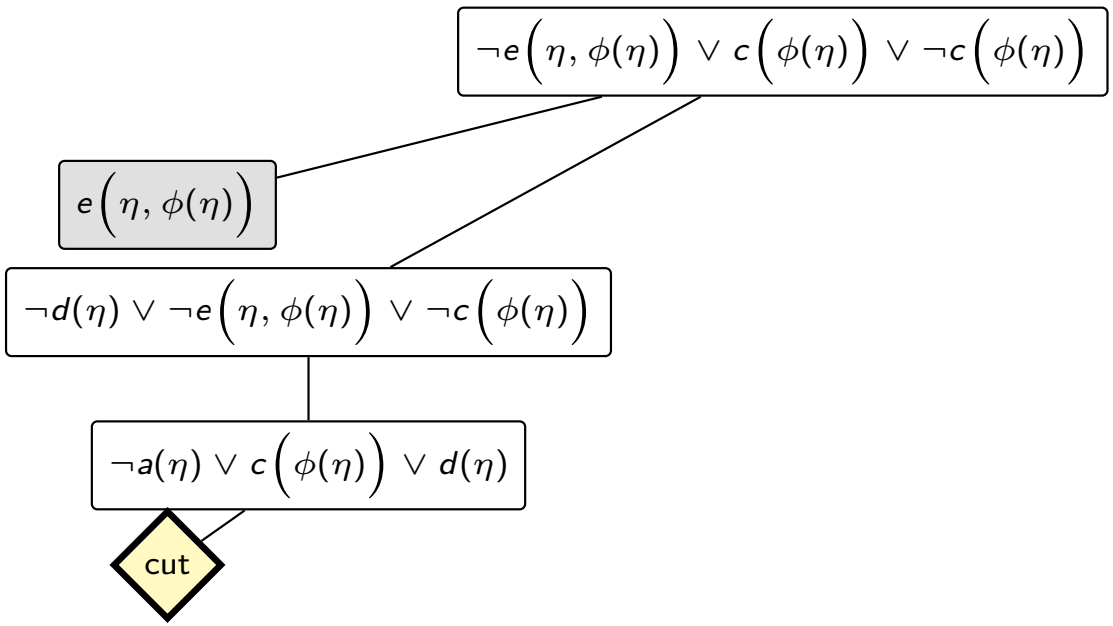

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



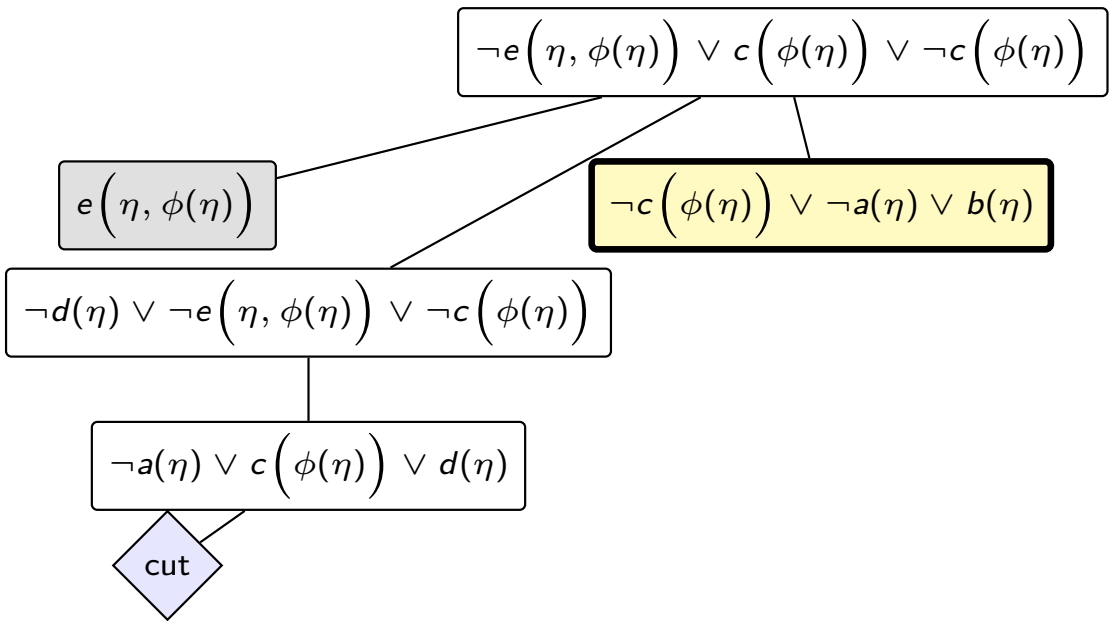

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



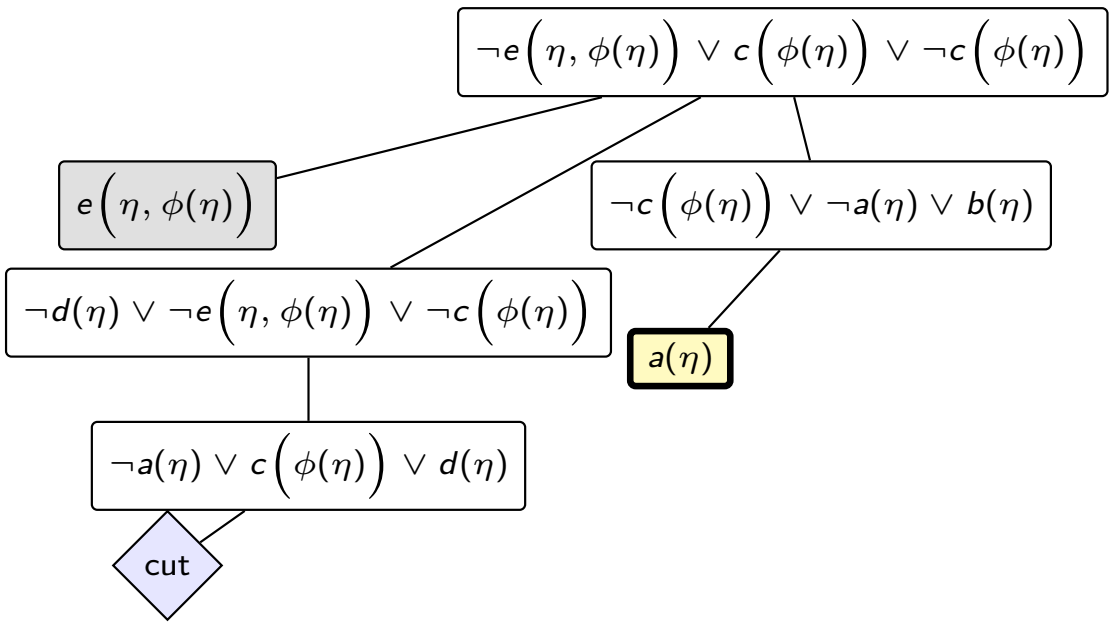

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



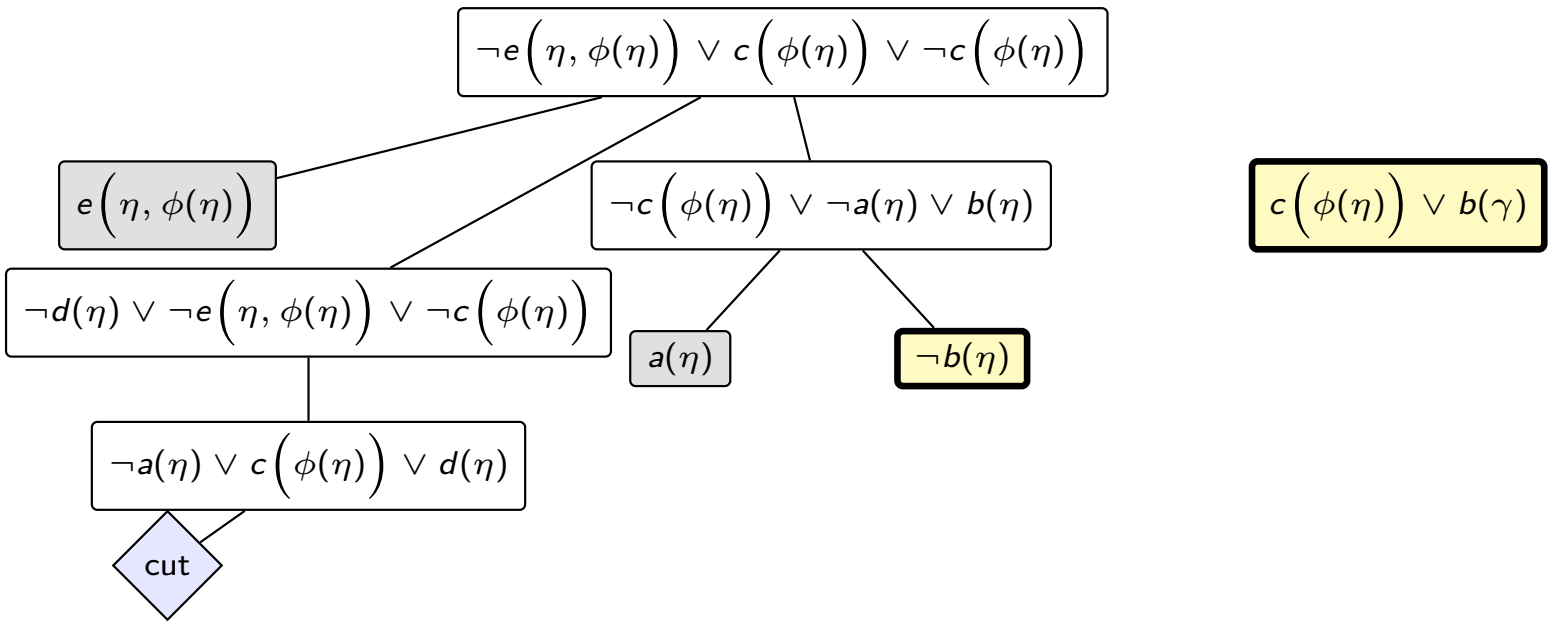

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



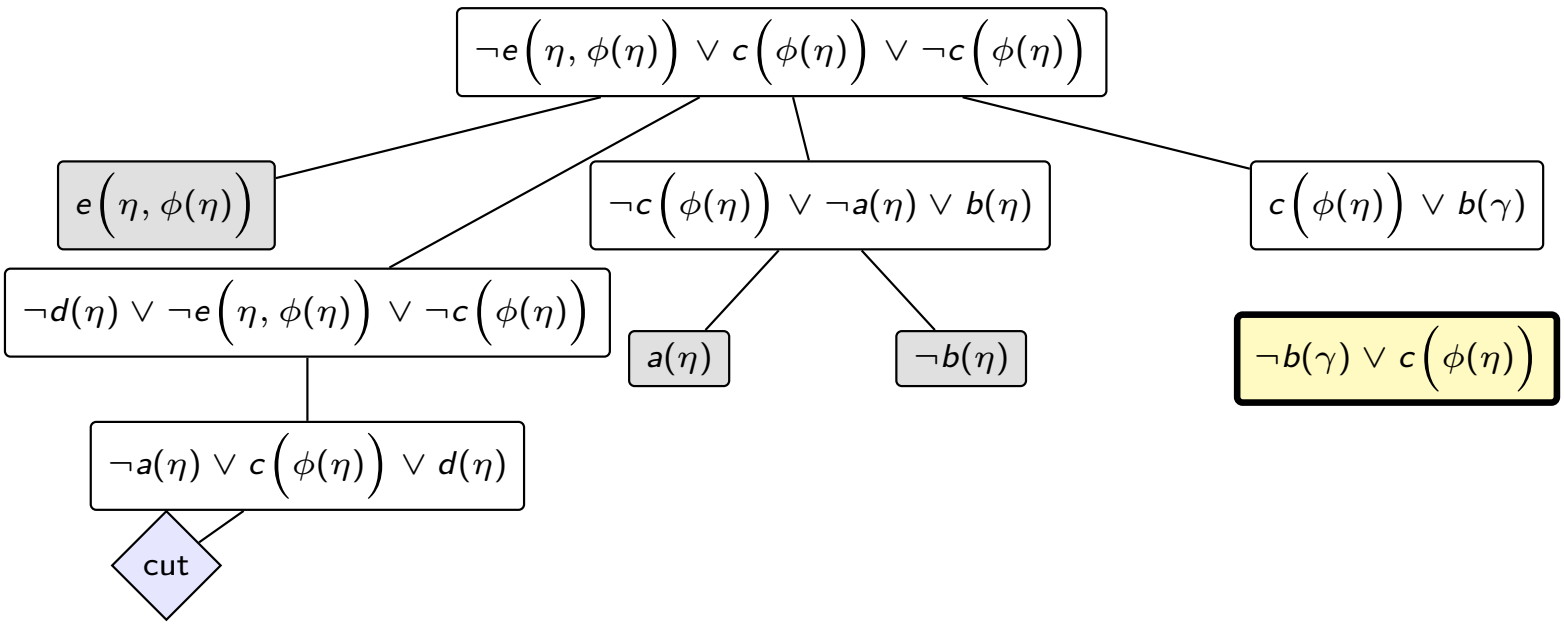

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP



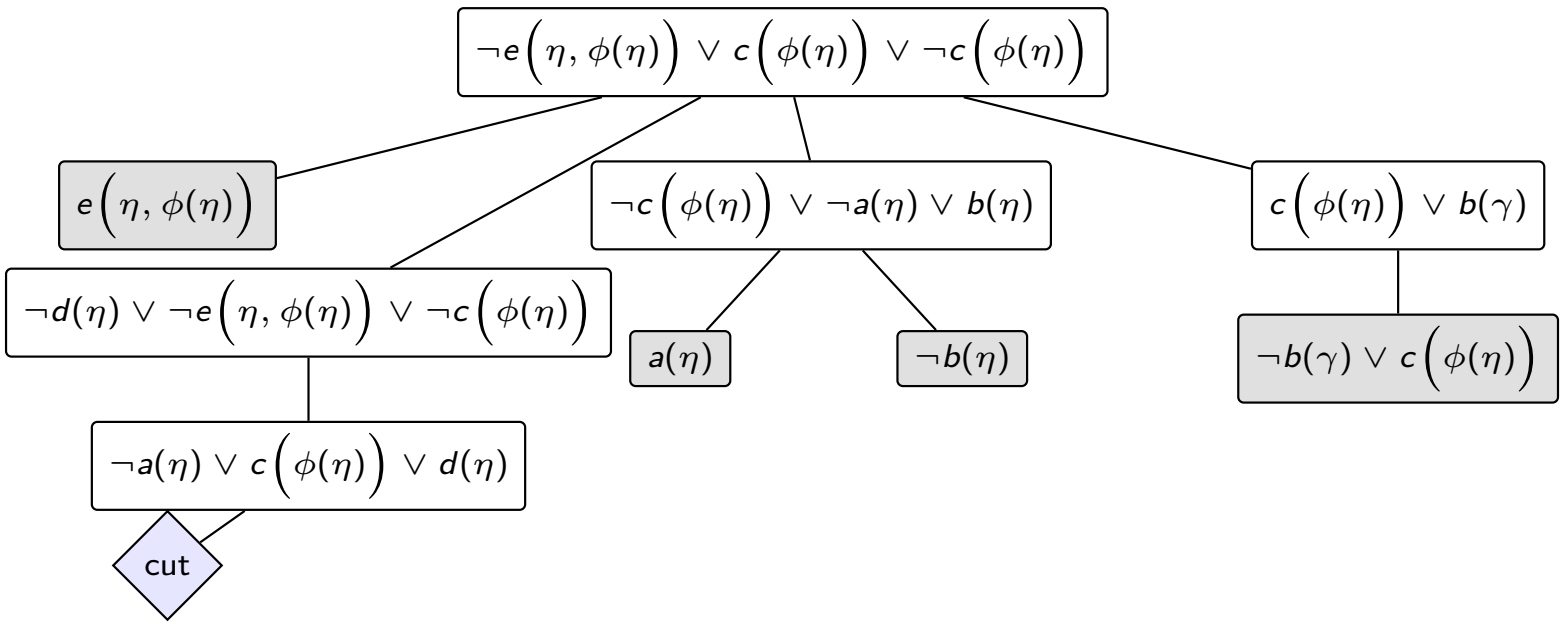

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP




Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

# leanCoP




total steps: 241

Otten and Bibel (2003): *leanCoP: Lean Connection-Based Theorem Proving*. *Journal of Symbolic Computation* 36(1–2)
Otten (2010): *Restricting Backtracking in Connection Calculi*. *AI Communications* 23(2–3)

UNIVERSITY OF CAMBRIDGE

# SATCoP

$a(\eta)$

persistent SAT ground

$a(\eta)$

# SATCoP

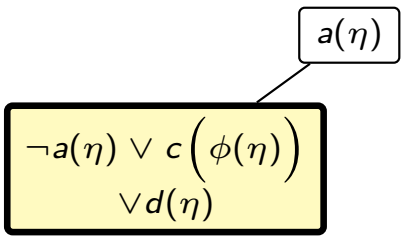
$a(\eta)$

persistent SAT ground

$a(\eta)$

UNIVERSITY OF CAMBRIDGE

# SATCoP

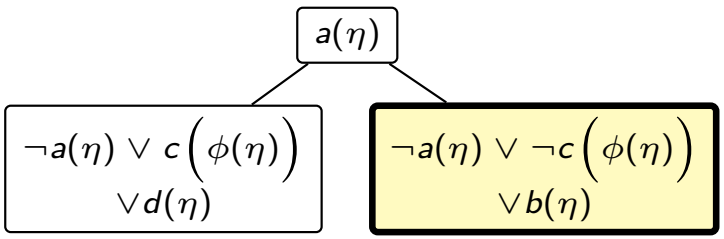



persistent SAT ground

$a(\eta)$

$\neg a(\eta) \vee c\Big(\phi(\eta)\Big) \vee d(\eta)$

# SATCoP



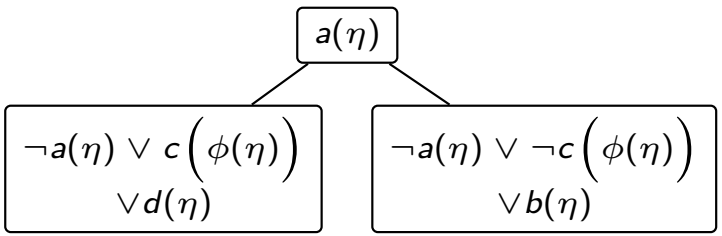

persistent SAT ground

$a(\eta)$

$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

# SATCoP



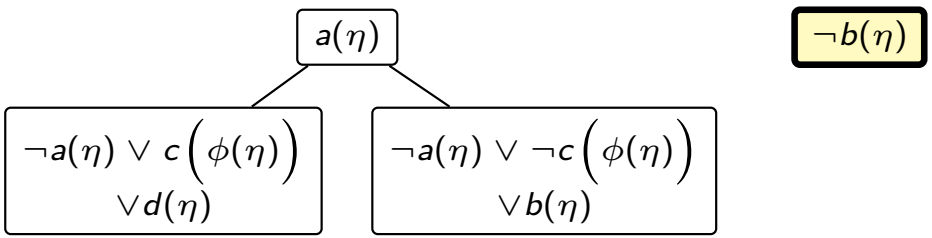

persistent SAT ground

$a(\eta)$

$\neg a(\eta) \vee c\Big(\phi(\eta)\Big) \vee d(\eta)$

$\neg a(\eta) \vee \neg c\Big(\phi(\eta)\Big) \vee b(\eta)$

# SATCoP




$\neg b(\eta)$

persistent SAT ground

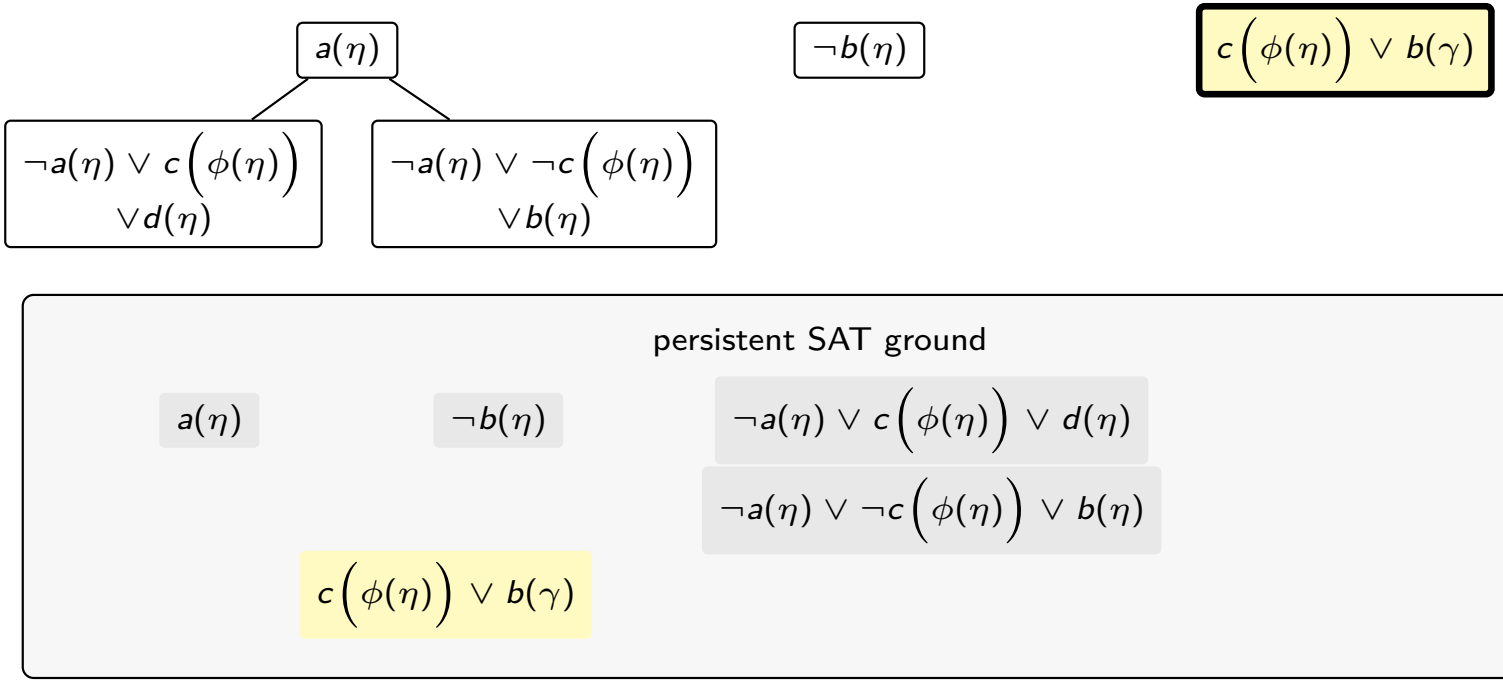
$a(\eta)$

$\neg b(\eta)$

$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

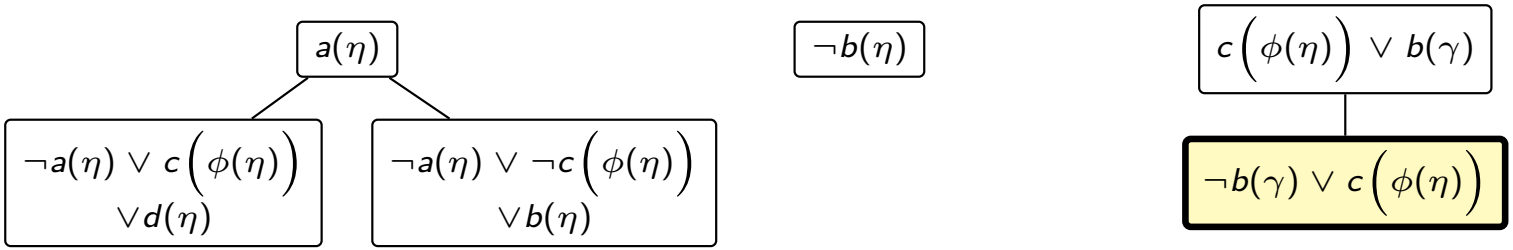
$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

# SATCoP


$a(\eta)$
$\neg b(\eta)$
$c\left(\phi(\eta)\right) \vee b(\gamma)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$
persistent SAT ground
$a(\eta)$
$\neg b(\eta)$
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$
$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$
$c\left(\phi(\eta)\right) \vee b(\gamma)$

UNIVERSITY OF CAMBRIDGE

# SATCoP




persistent SAT ground

$a(\eta)$

$\neg b(\eta)$

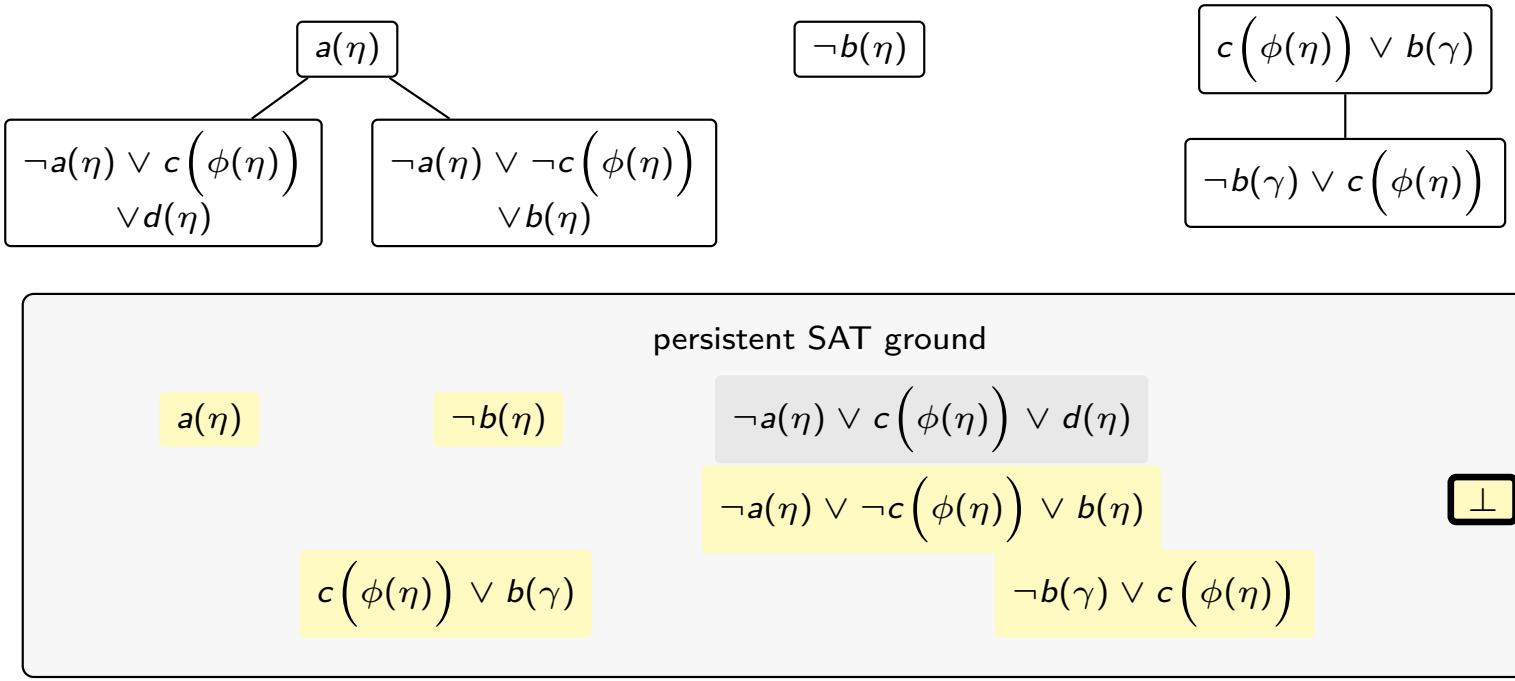
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

# SATCoP




total steps: 93

- **SATCoP$_0$** uses the SAT solver to check whether the ground clauses are inconsistent.
- **SATCoP** also looks at the current SAT model.
- When choosing the next connection, it tries a goal that is true in that model. If the model gives no preference, it keeps the shuffled order.

UNIVERSITY OF CAMBRIDGE

# SATCoP

$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

persistent SAT ground

$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

UNIVERSITY OF CAMBRIDGE

# SATCoP

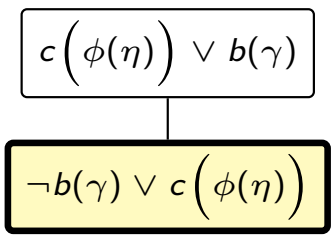
$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

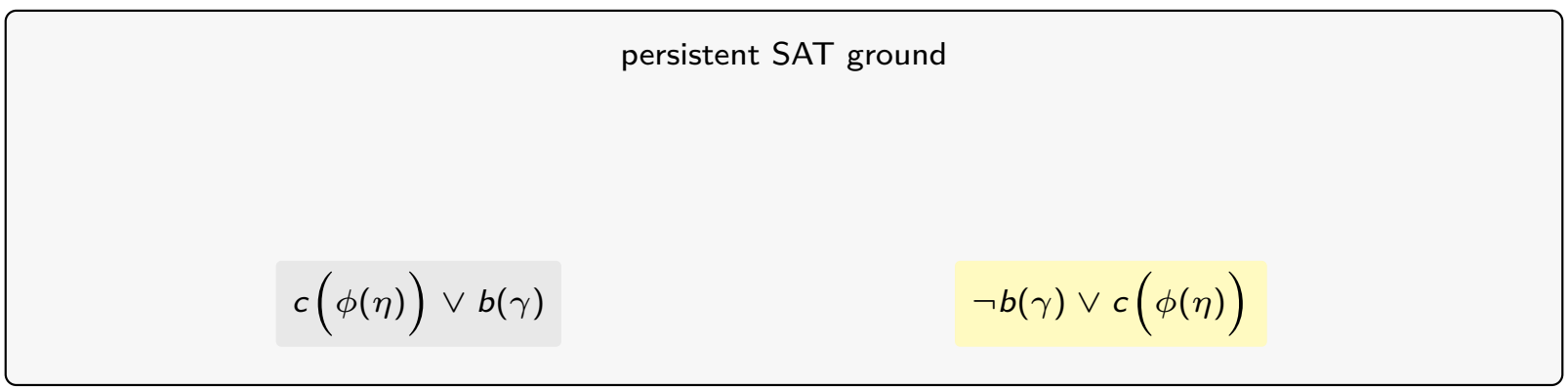
persistent SAT ground

$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

UNIVERSITY OF CAMBRIDGE

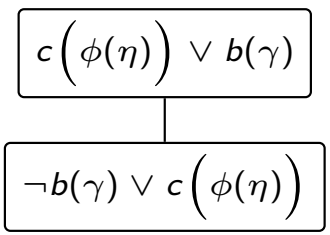
# SATCoP

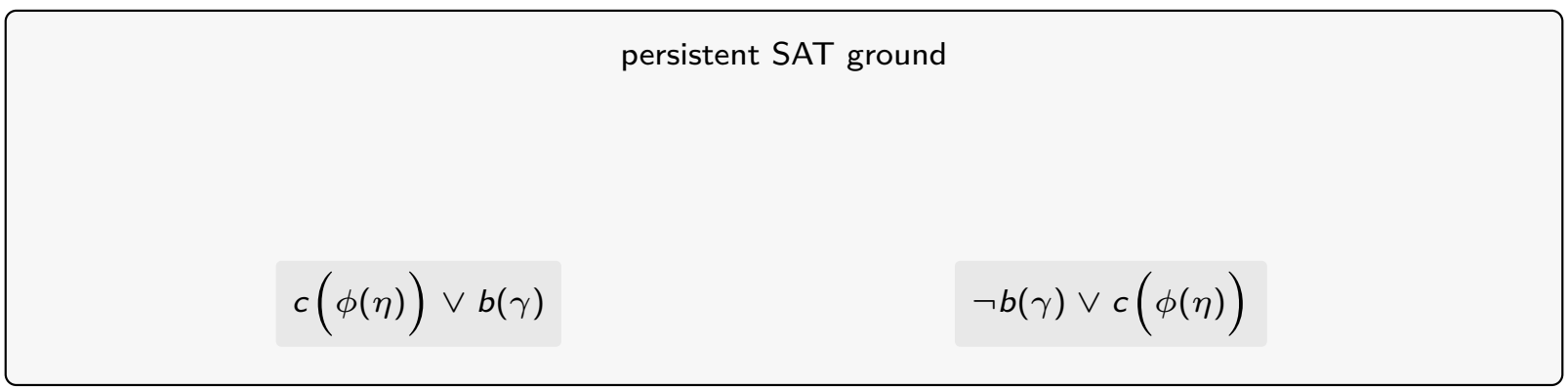


UNIVERSITY OF CAMBRIDGE

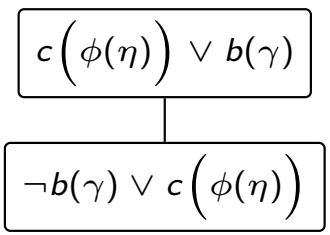
# SATCoP



UNIVERSITY OF CAMBRIDGE

# SATCoP




$a(\eta)$

persistent SAT ground

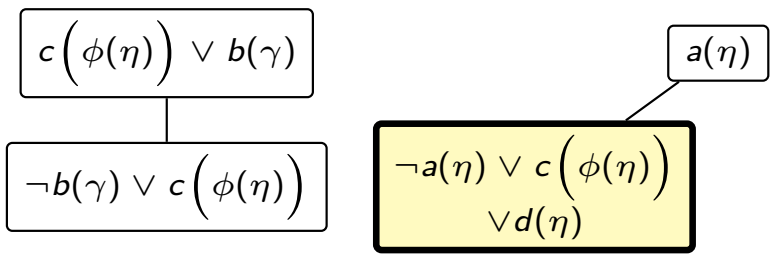
$a(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

UNIVERSITY OF CAMBRIDGE

# SATCoP




persistent SAT ground

$a(\eta)$

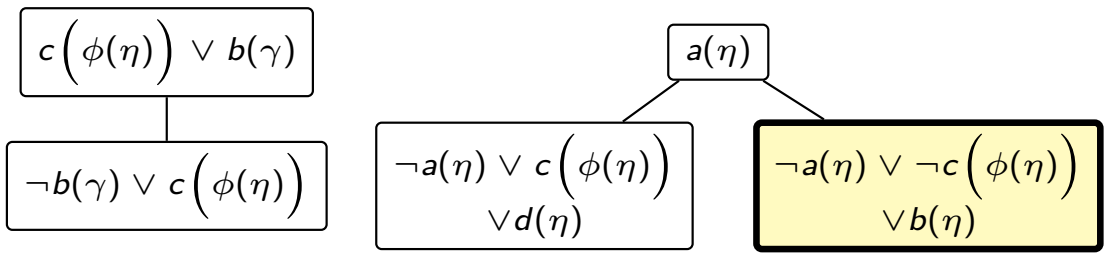
$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

UNIVERSITY OF CAMBRIDGE

# SATCoP




persistent SAT ground

$a(\eta)$

$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

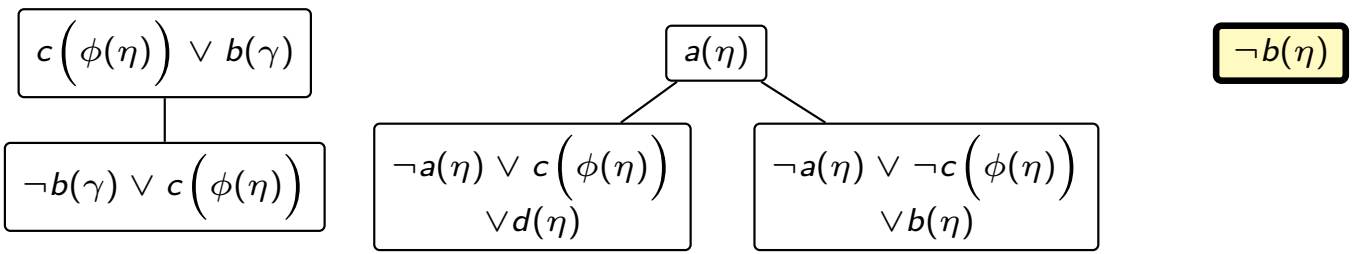
$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

UNIVERSITY OF CAMBRIDGE

# SATCoP




persistent SAT ground

$a(\eta)$

$\neg b(\eta)$

$\neg a(\eta) \vee c\left(\phi(\eta)\right) \vee d(\eta)$

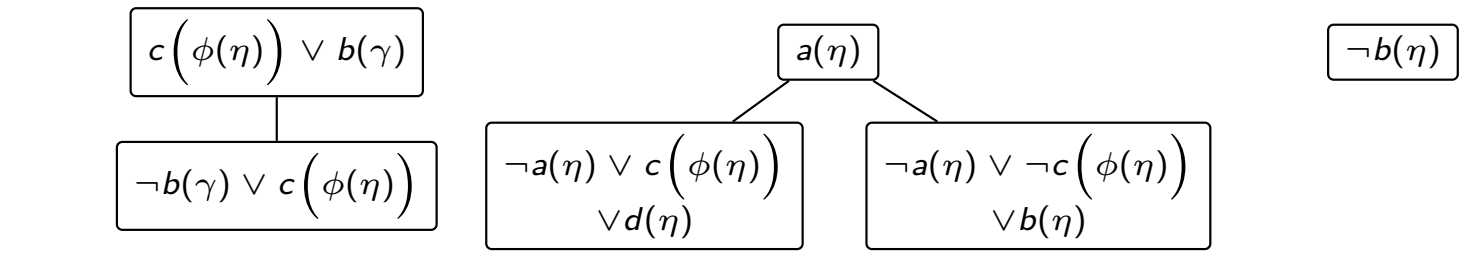
$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

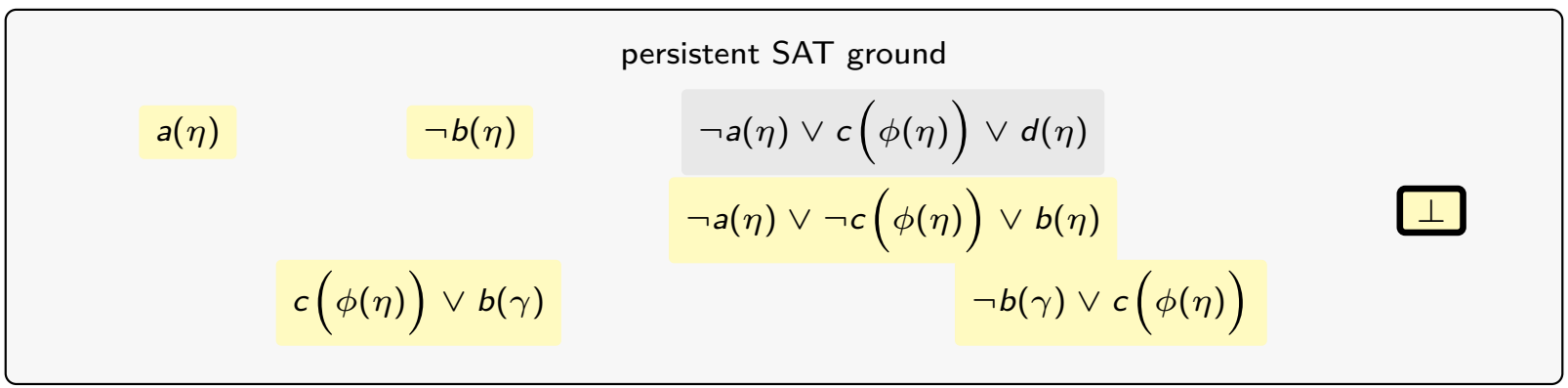
$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

UNIVERSITY OF CAMBRIDGE

# SATCoP




total steps: 75

UNIVERSITY OF CAMBRIDGE

**Connection tableau**

Performs goal-directed search for relevant ground instances.

+

**Persistent SAT solver**

Retains those instances and detects propositional **UNSAT**.

A failed tableau can still contribute to the proof.

- Keep newly generated ground clauses.
- **Reset** the tableau instead of locally backtracking.
- Increase the depth only when an attempt produces nothing new.

SAT refutations remain **sound**; the reset search is **incomplete**.

Our objective is to **saturate the SAT shadow**.

Our objective is to **saturate the SAT shadow**.

We borrow the idea of iterative deepening from leanCoP 2.0.

- New SAT propositional entries on this tree? Keep the same height.
- No new SAT propositional entries? `max_height += 1`.

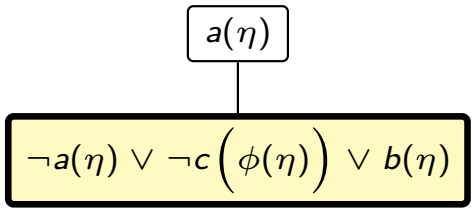
# SATResetCoP

$a(\eta)$

persistent SAT ground

$a(\eta)$

# SATResetCoP

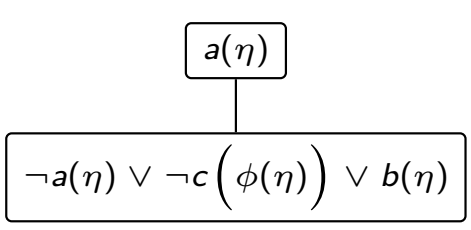



persistent SAT ground

$a(\eta)$

$\neg a(\eta) \vee \neg c\Big(\phi(\eta)\Big) \vee b(\eta)$

# SATResetCoP




$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

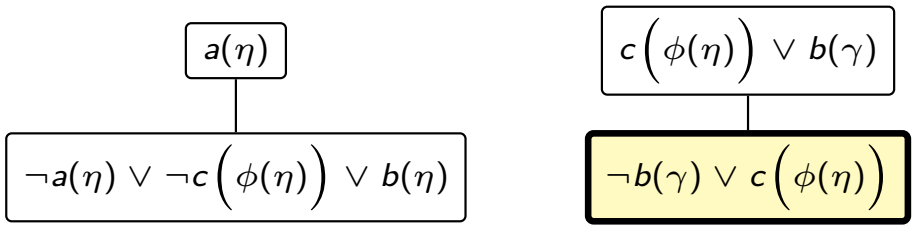
persistent SAT ground

$a(\eta)$

$$\neg a(\eta) \vee \neg c\Big(\phi(\eta)\Big) \vee b(\eta)$$

$$c\Big(\phi(\eta)\Big) \vee b(\gamma)$$

# SATResetCoP




persistent SAT ground

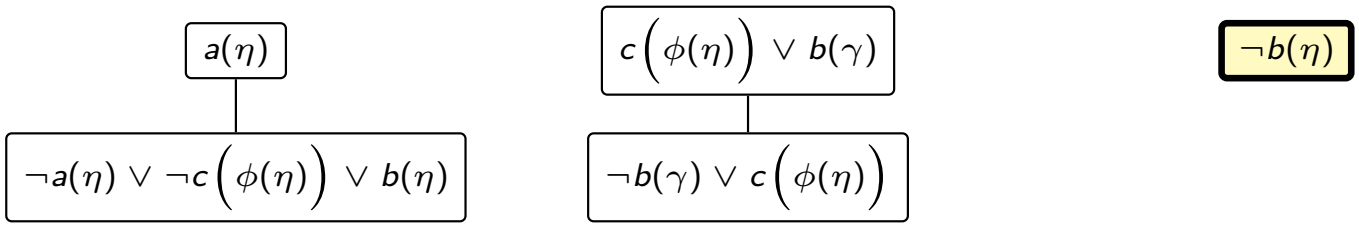
$a(\eta)$

$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

# SATResetCoP




persistent SAT ground

$a(\eta)$

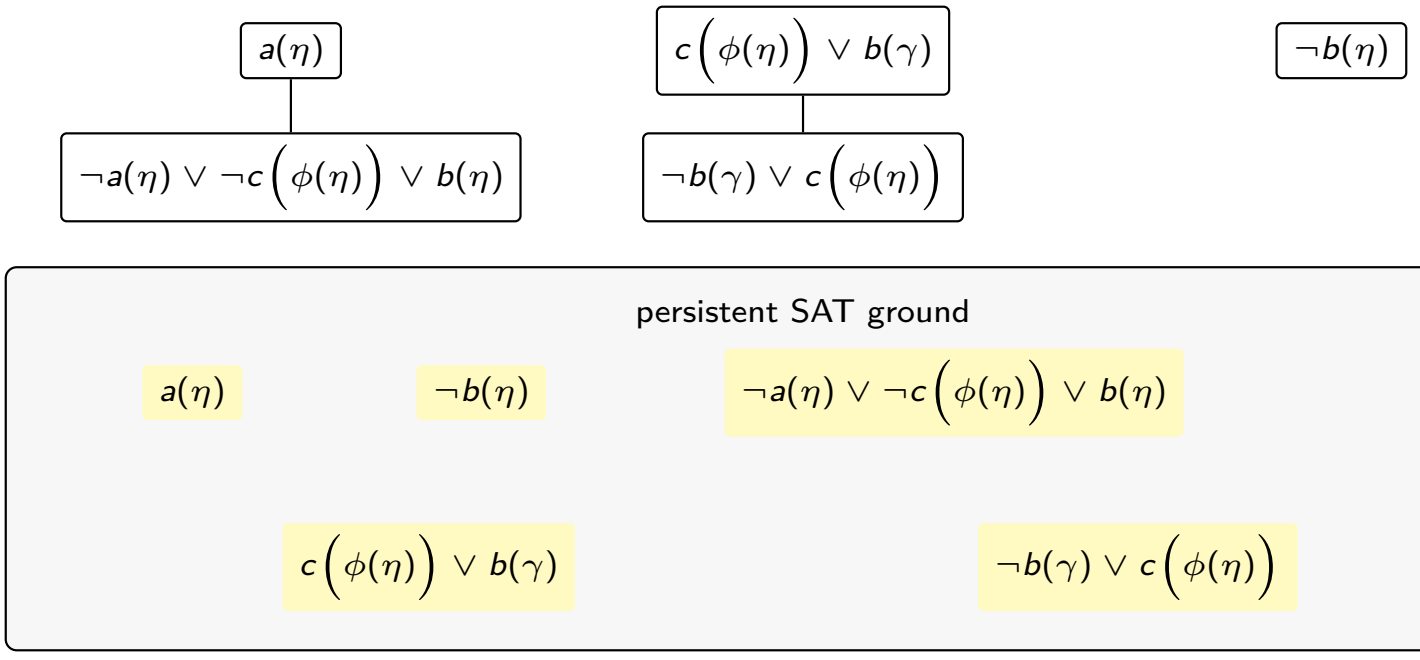
$\neg b(\eta)$

$\neg a(\eta) \vee \neg c\left(\phi(\eta)\right) \vee b(\eta)$

$c\left(\phi(\eta)\right) \vee b(\gamma)$

$\neg b(\gamma) \vee c\left(\phi(\eta)\right)$

# SATResetCoP




total steps: 10

SYN081-1.p

$$\neg p(X) \vee \neg p(\psi(X))$$
$$p(\psi(X)) \vee p(X)$$
$$\neg p(X) \vee p(\psi(X))$$

$p(X)$ an arbitrary unary atom.

$\psi(X)$ an arbitrary unary function.

$X$ a variable, universally quantified in every clause.

$\kappa$ the fresh constant used to ground clauses for the SAT shadow.

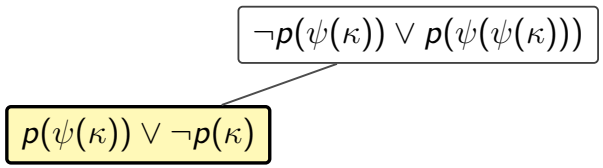
# SATCoP

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

# SATCoP

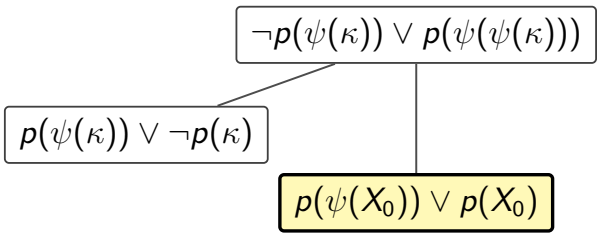



**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

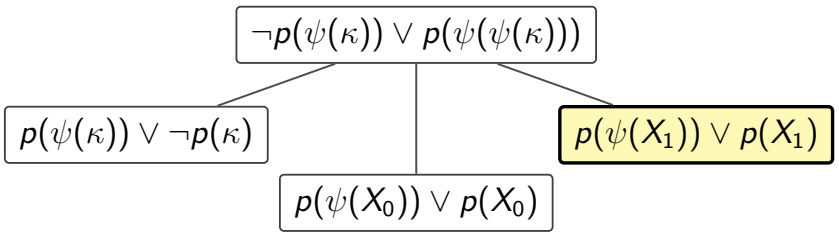
# SATCoP




**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

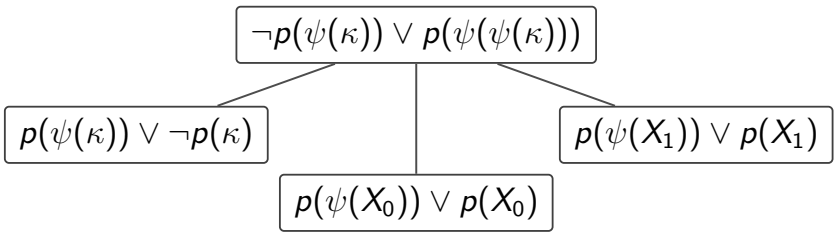
# SATCoP




**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

# SATCoP

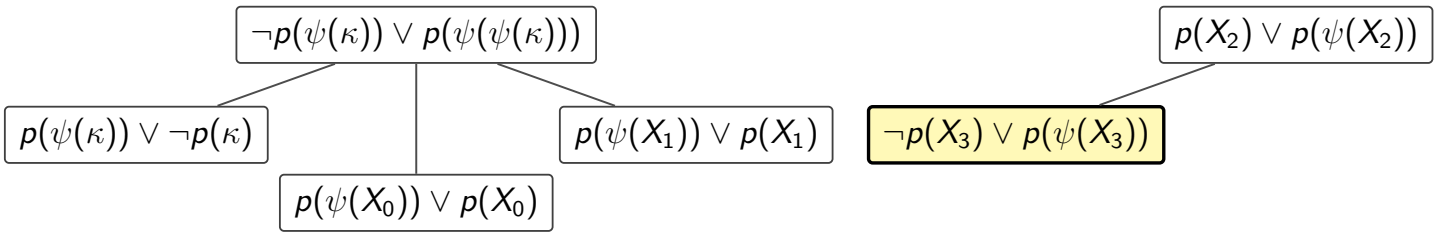



$p(X_2) \vee p(\psi(X_2))$

**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

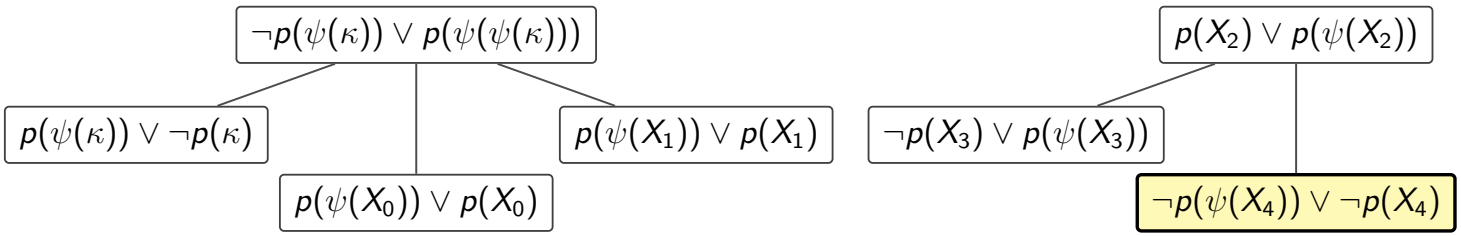
# SATCoP




**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

# SATCoP




**persistent SAT ground**

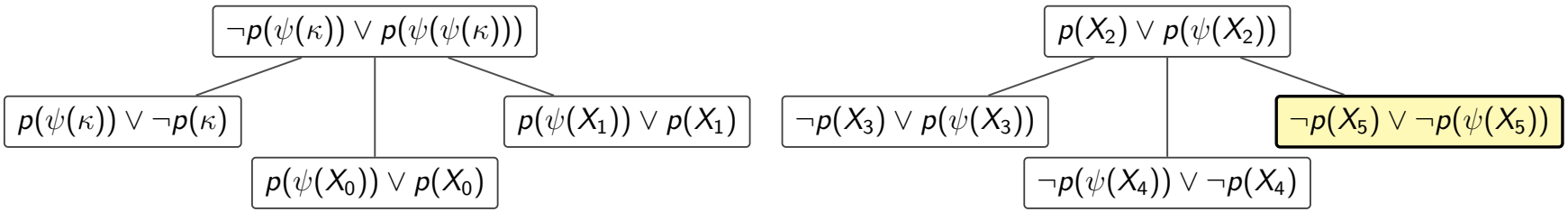
$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP




**persistent SAT ground**

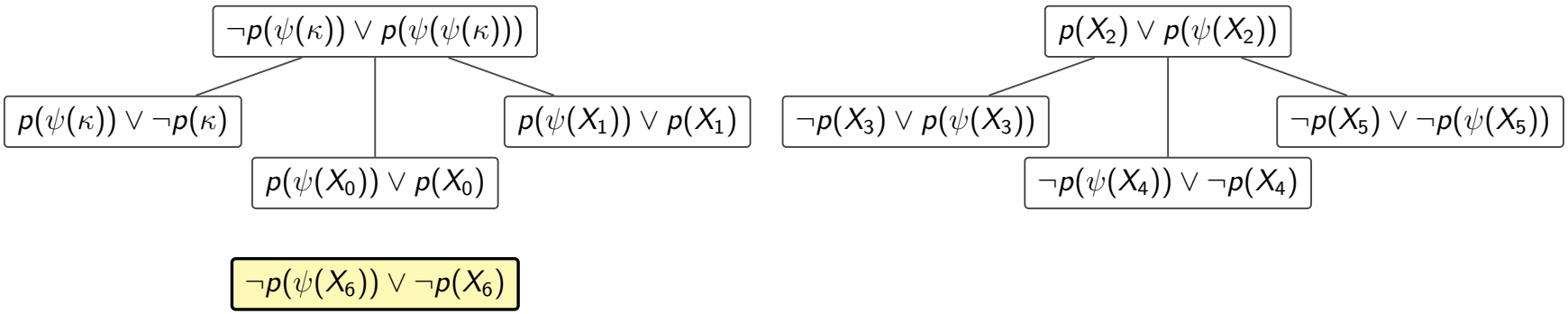
$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP




**persistent SAT ground**

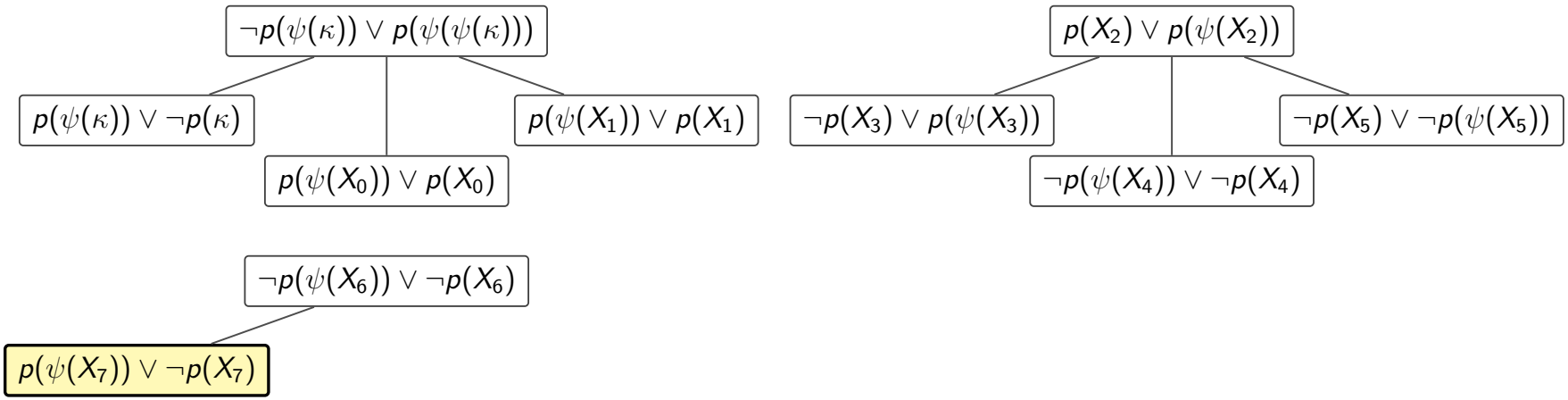
$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP

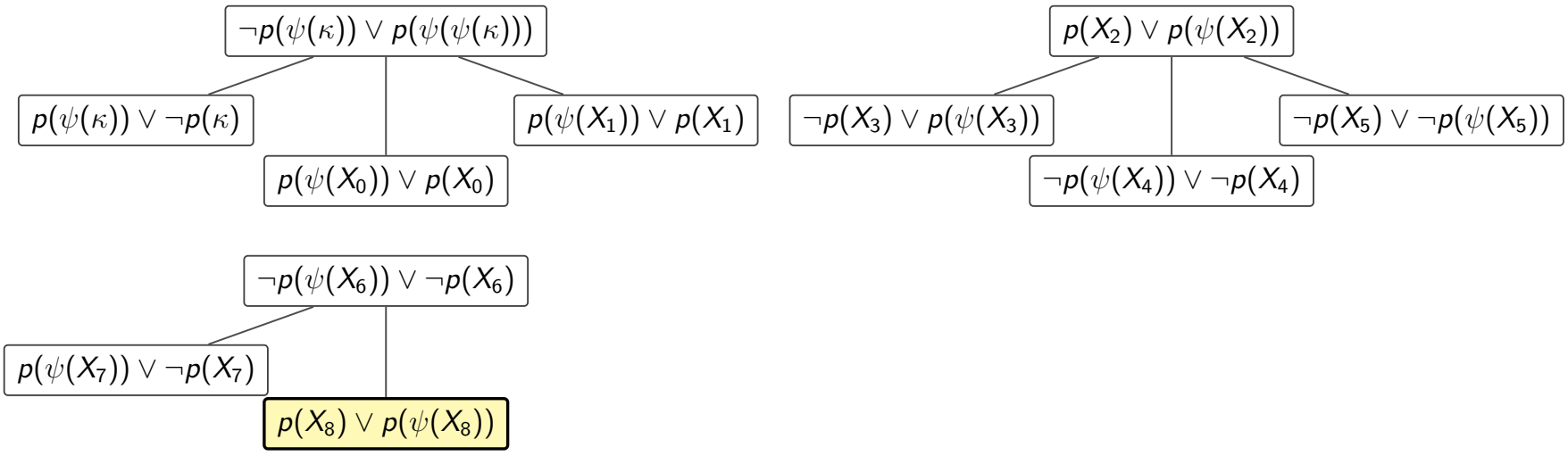



**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP

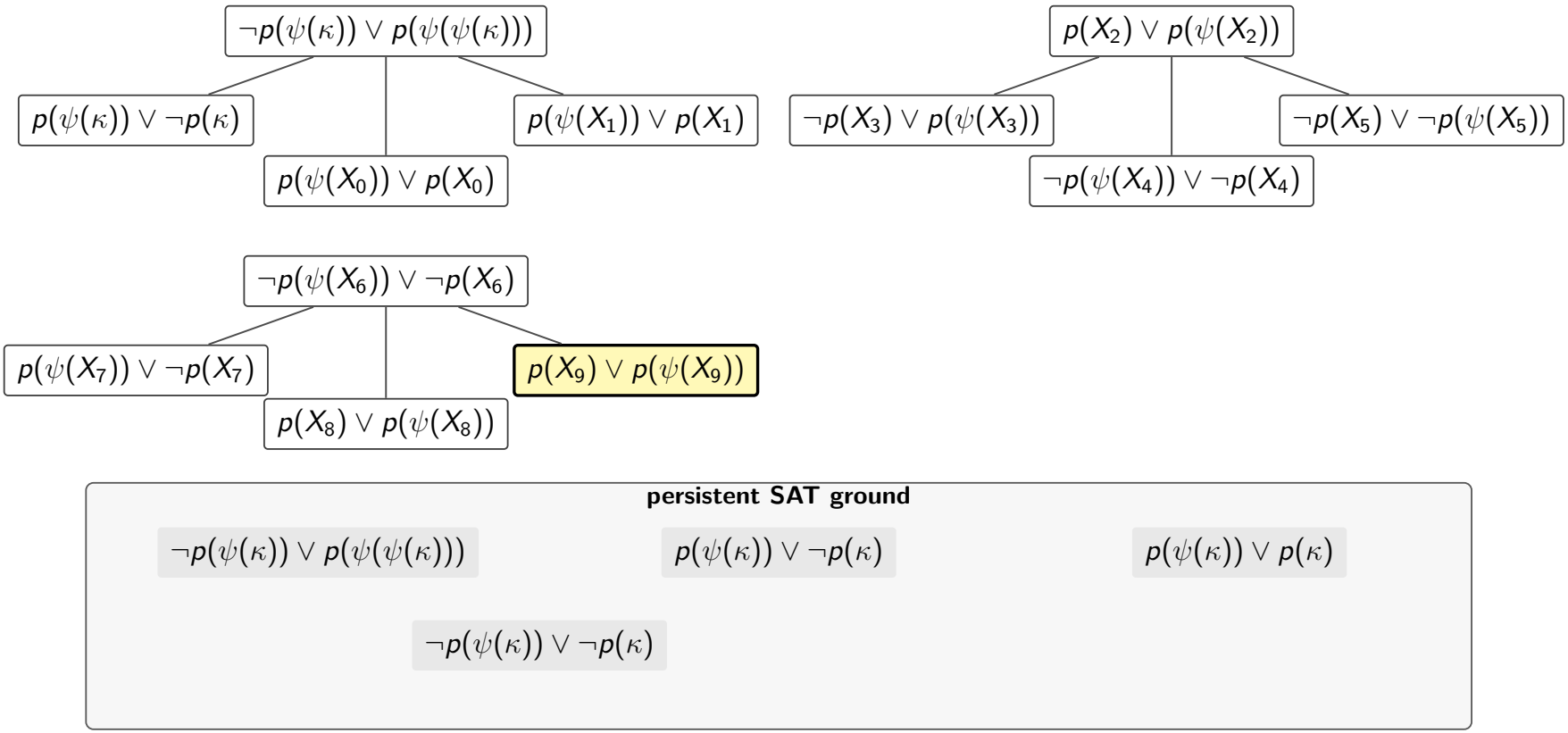



**persistent SAT ground**

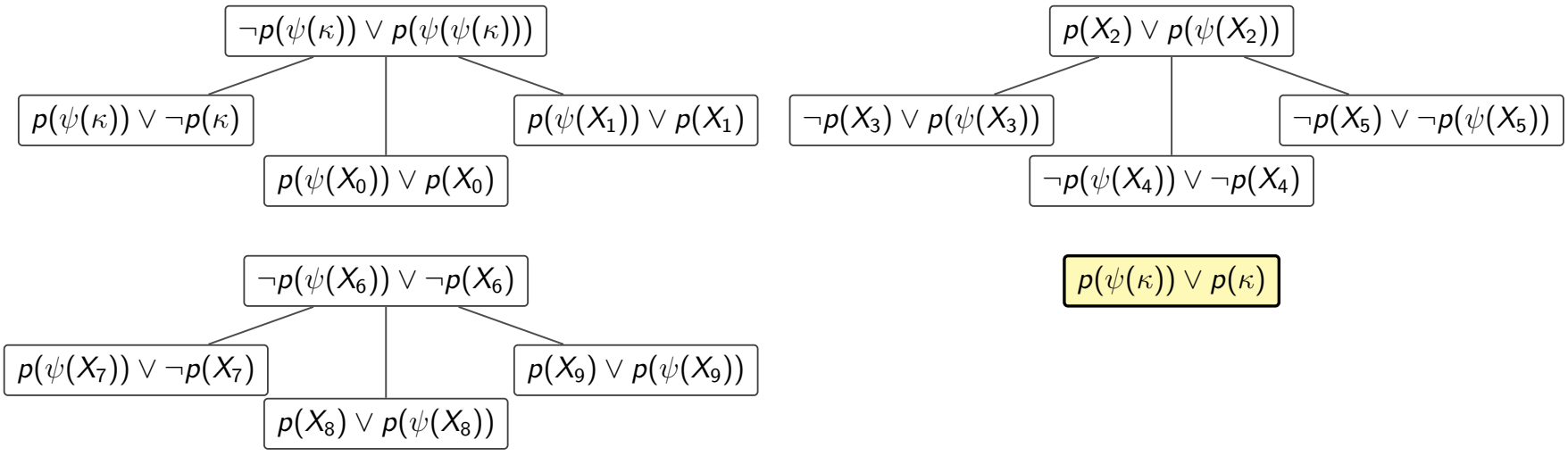
$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP


$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$
$p(\psi(\kappa)) \vee \neg p(\kappa)$
$p(\psi(X_1)) \vee p(X_1)$
$p(\psi(X_0)) \vee p(X_0)$
$p(X_2) \vee p(\psi(X_2))$
$\neg p(X_3) \vee p(\psi(X_3))$
$\neg p(X_5) \vee \neg p(\psi(X_5))$
$\neg p(\psi(X_4)) \vee \neg p(X_4)$
$\neg p(\psi(X_6)) \vee \neg p(X_6)$
$p(\psi(X_7)) \vee \neg p(X_7)$
$p(X_9) \vee p(\psi(X_9))$
$p(X_8) \vee p(\psi(X_8))$
persistent SAT ground
$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$
$p(\psi(\kappa)) \vee \neg p(\kappa)$
$p(\psi(\kappa)) \vee p(\kappa)$
$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP

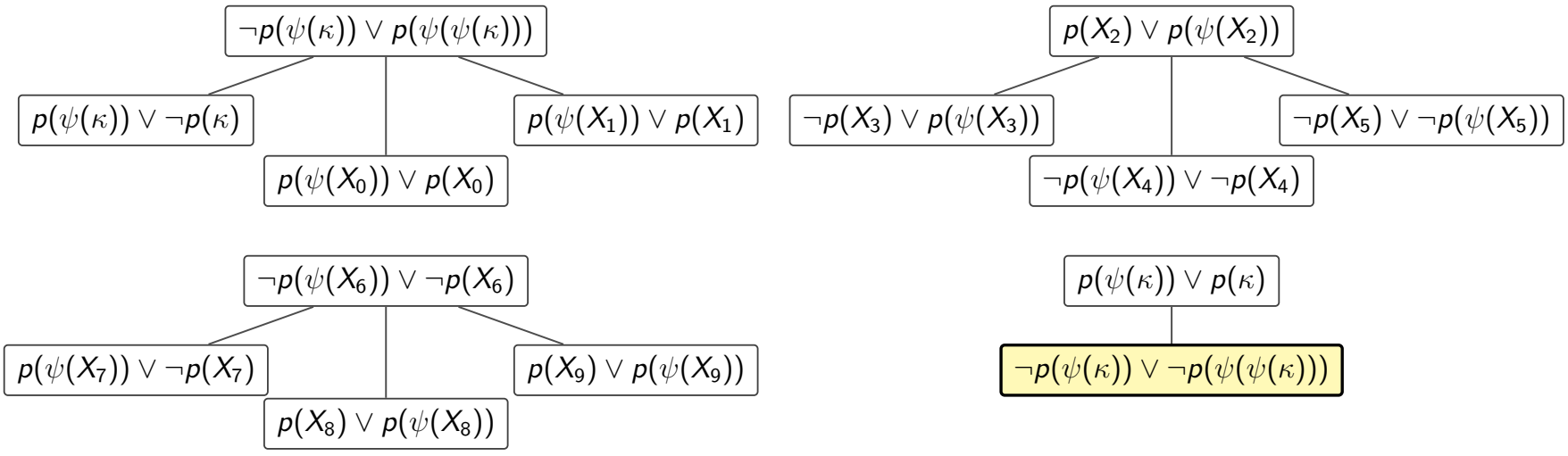



**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATCoP

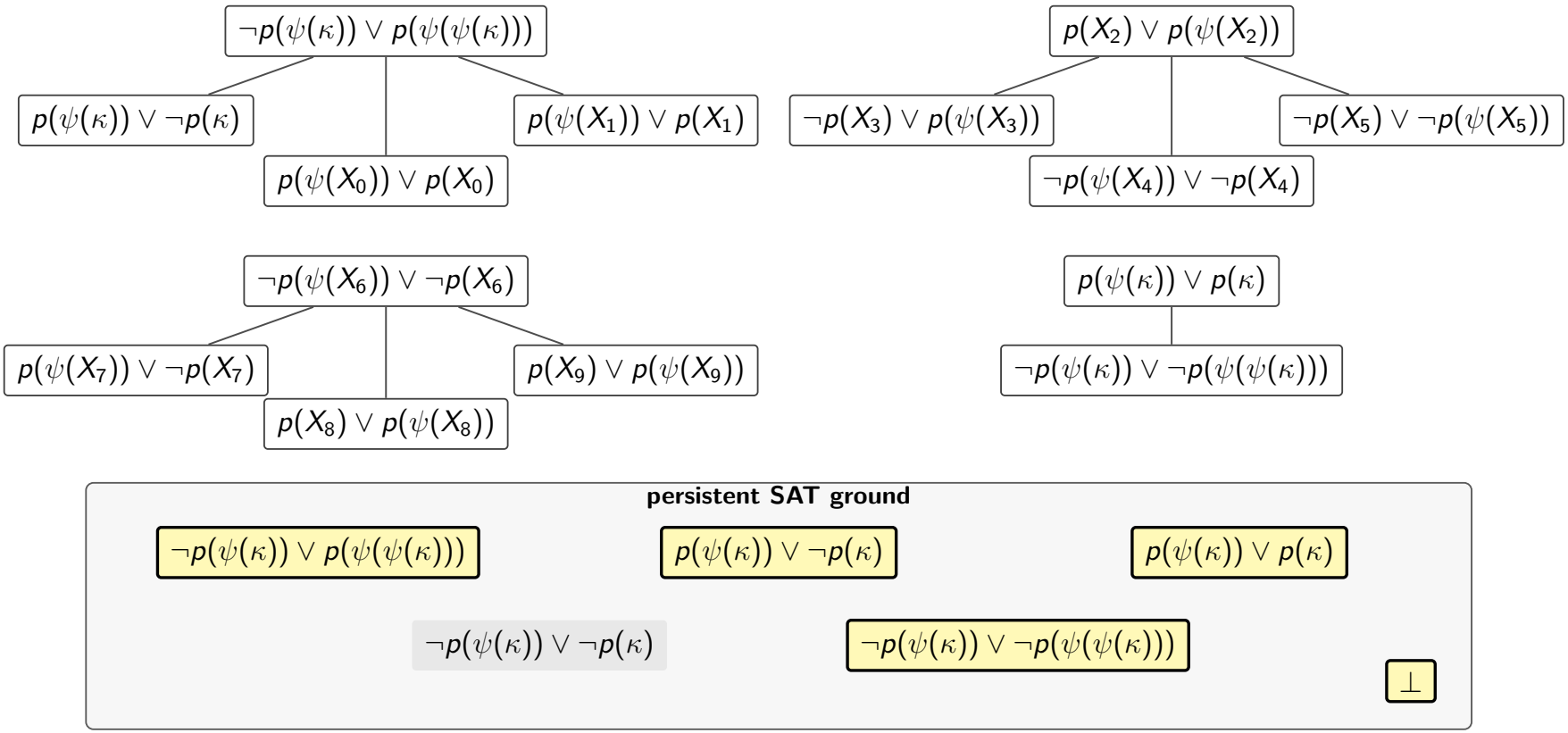



**persistent SAT ground**

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$ $p(\psi(\kappa)) \vee \neg p(\kappa)$ $p(\psi(\kappa)) \vee p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\kappa)$ $\neg p(\psi(\kappa)) \vee \neg p(\psi(\psi(\kappa)))$

# SATCoP




total steps: 24

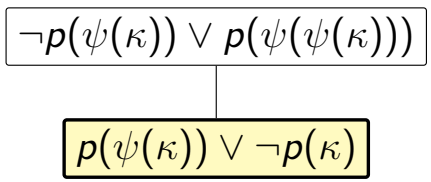
# SATResetCoP

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

persistent SAT ground

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

# SATResetCoP




persistent SAT ground

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

# SATResetCoP




persistent SAT ground

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\psi(\psi(\kappa)))$

# SATResetCoP




persistent SAT ground

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee p(\kappa)$

# SATResetCoP




persistent SAT ground

$\neg p(\psi(\kappa)) \vee p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee \neg p(\kappa)$

$\neg p(\psi(\kappa)) \vee \neg p(\psi(\psi(\kappa)))$

$p(\psi(\kappa)) \vee p(\kappa)$

$\bot$

total steps: 5

# Experimental Evaluation

| Strategy | Implementation | Cut | SAT refutation | SAT guidance | **Reset** on **backtrack** |
|---|---|---|---|---|---|
| **leanCoP** | Prolog | ✓ | | | |
| **SATCoP$_0$** | Rust | ✓ | ✓ | | |
| **SATCoP** | Rust | ✓ | ✓ | ✓ | |
| **SATResetCoP** | Python | ✓ | ✓ | ✓ | ✓ |

| Strategy | Implementation | Cut | SAT refutation | SAT guidance | **Reset** on **backtrack** |
|---|---|---|---|---|---|
| **leanCoP$^{con}$** | Python | ✓ | | | |
| **SATCoP$_0$$^{con}$** | Python | ✓ | ✓ | | |
| **SATCoP$^{con}$** | Python | ✓ | ✓ | ✓ | |
| **SATResetCoP$^{con}$** | Python | ✓ | ✓ | ✓ | ✓ |

UNIVERSITY OF CAMBRIDGE

## Connections Library

Connections is a Python library for building connection-tableau provers. It supplies representation of connection tableaux.
We implement **leanCoP$^{con}$**, **SATCoP$_0{}^{con}$**, **SATCoP$^{con}$**, and **SATResetCoP** as different policies over this same backend.

**CaDiCaL 2.0**

All SAT-based policies use the same incremental solver to retain and test the accumulated ground clauses.

## Cambridge CSD3

**Ice Lake node**

Intel Xeon Platinum 8368Q at 2.60 GHz

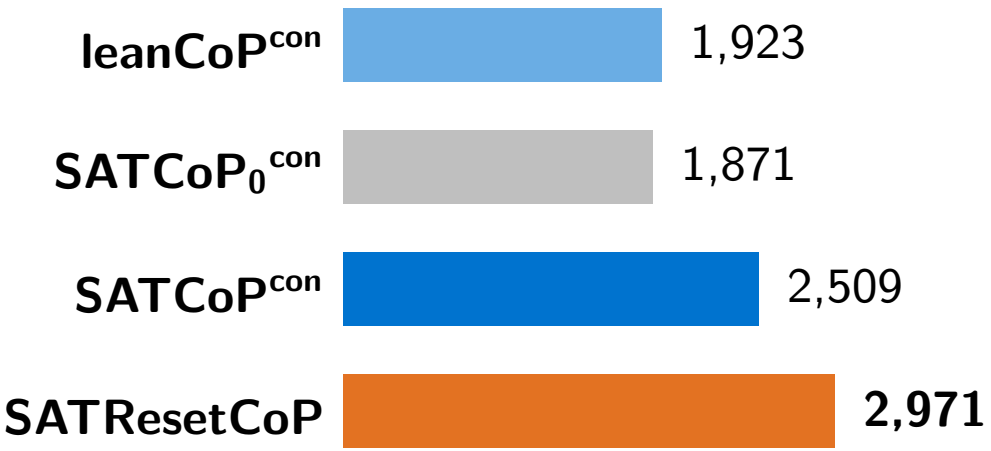
- dual socket, 38 cores each
- one hardware thread per core
- 3,370 MiB RAM per core
- each run uses one core

Rømming et al. (2023): *Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi*. *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, CEUR Workshop Proceedings vol. 3613
Biere et al. (2024): *CaDiCaL 2.0*. *Computer Aided Verification*, Lecture Notes in Computer Science vol. 14681
University of Cambridge Research Computing Services (2026): *Ice Lake Nodes*

UNIVERSITY OF CAMBRIDGE

**Limits:** 120 seconds or 1,000,000 steps per problem

**TPTP problems solved**




**2,971**
problems solved

**+462**
vs. **SATCoP (+18%)**

**+1,048**
vs. **leanCoP (+54%)**

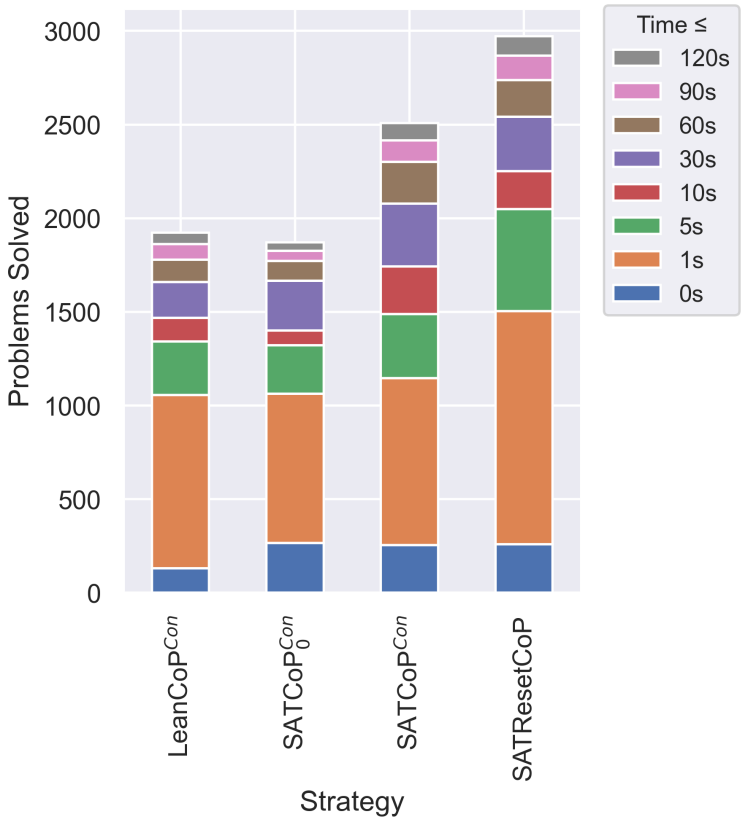
**Aggressive resets solve 18% more TPTP problems than the best SATCoP baseline.**

Sutcliffe (2017): *The TPTP Problem Library and Associated Infrastructure: From CNF to TH0, TPTP v6.4.0.* *Journal of Automated Reasoning* 59(4)

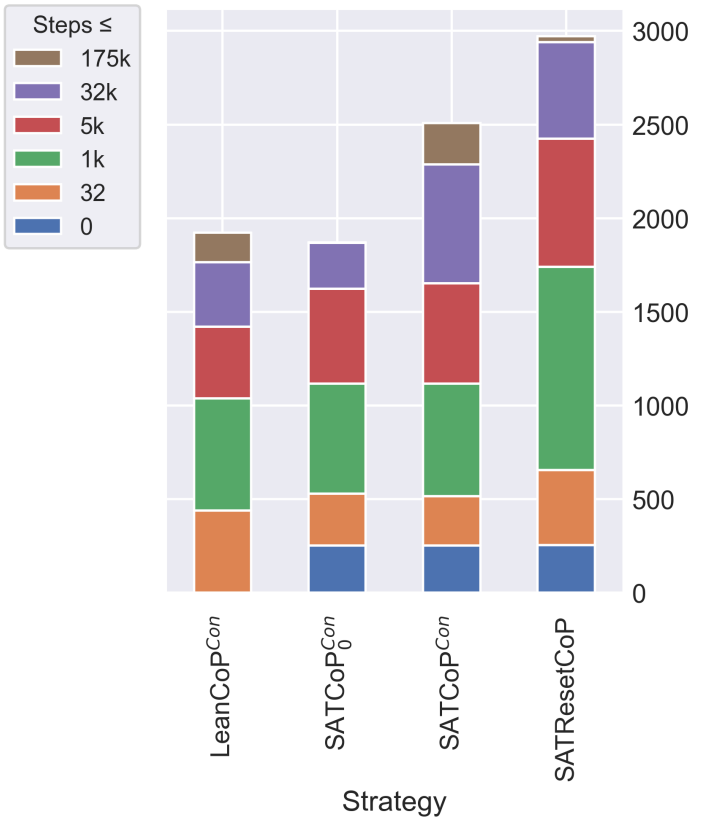
# Faster Proving = More Proofs

UNIVERSITY OF CAMBRIDGE

## Solved by wall-clock time



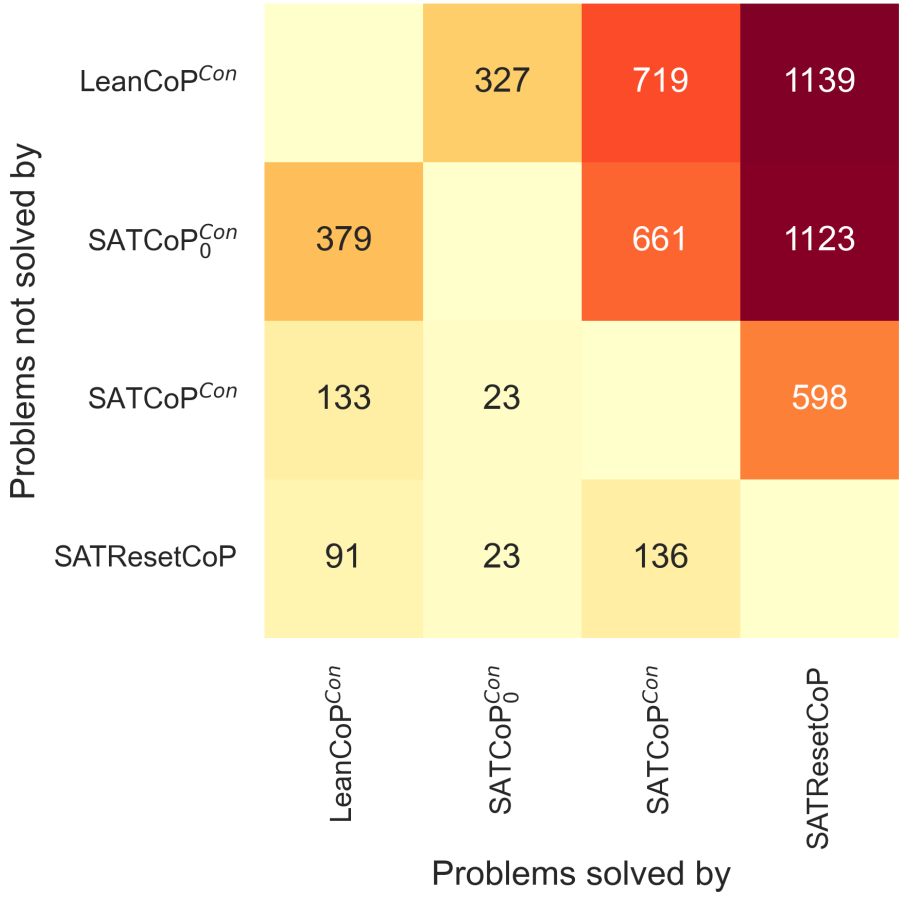

## Solved by tableau steps




**Resetting reaches more SAT refutations with less tableau search.**

A 0-step proof means that the initial ground clause set was already propositionally unsatisfiable.

# Resets Change the Search




Aggressive resets explore a different part of the search space—a schedule should retain both modes.

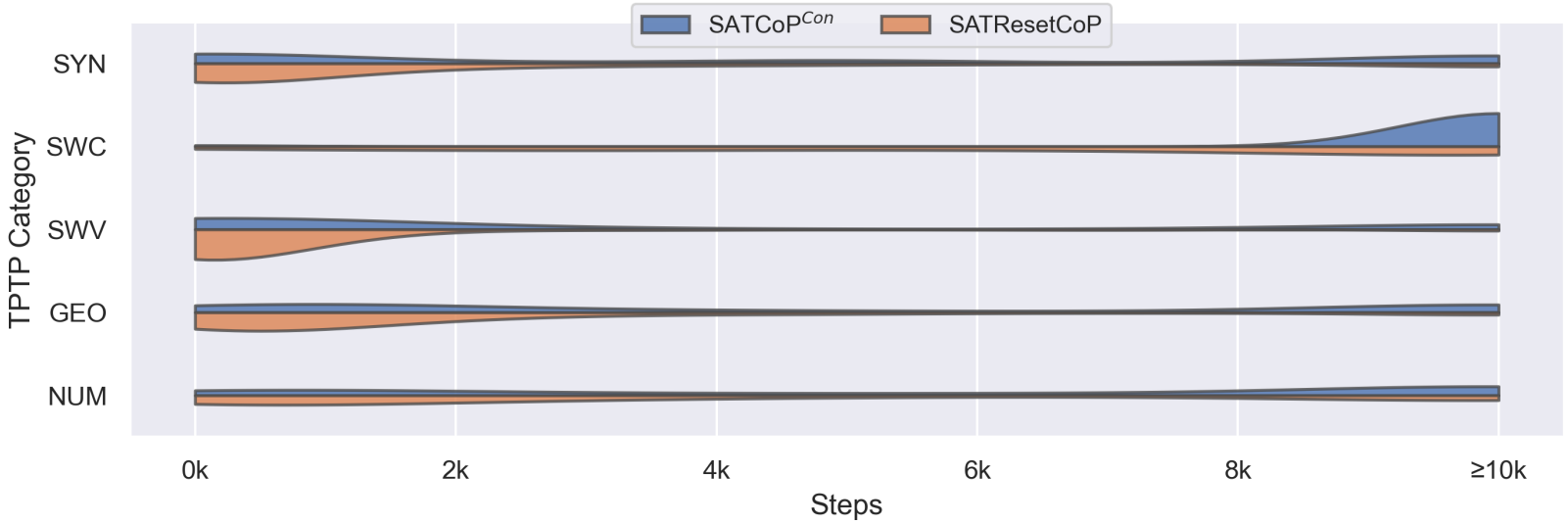
# Where **Reset** Helps

UNIVERSITY OF CAMBRIDGE

**Top five absolute gains over SATCoP**

| | SATCoP | SATResetCoP | Gain |
|---|---|---|---|
| SYN | 549 | 672 | **+123** |
| SWC | 11 | 64 | **+53** |
| SWV | 173 | 215 | **+42** |
| GEO | 260 | 293 | **+33** |
| NUM | 151 | 179 | **+28** |

Useful ground instances appear early, so retained clauses reach propositional **UNSAT** before local tableau search pays off.




Step distributions on the five categories above; unsolved problems are placed in $\geq$ 10k.

# Takeaways

**Keep the clauses, reset the tree.**

- Aggressive resets help, but change which problems are solved.
- Schedule **Reset** search before conventional **SATCoP** or **leanCoP** search.



Mangla et al. (2022): *Bayesian Ranking for Strategy Scheduling in Automated Theorem Provers*. *International Joint Conference on Automated Reasoning (IJCAR)*
Biere et al. (2024): *CaDiCaL 2.0*. *Computer Aided Verification*, Lecture Notes in Computer Science vol. 14681

**Keep the clauses, reset the tree.**

- Aggressive resets help, but change which problems are solved.
- Schedule **Reset** search before conventional **SATCoP** or **leanCoP** search.
- **See you at CASC on Monday!**


Mangla et al. (2022): *Bayesian Ranking for Strategy Scheduling in Automated Theorem Provers*. *International Joint Conference on Automated Reasoning (IJCAR)*
Biere et al. (2024): *CaDiCaL 2.0*. *Computer Aided Verification*, Lecture Notes in Computer Science vol. 14681

# Acknowledgements

Thanks to Fredrik Rømming for his prior work on the Connections Prover.

This work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service (`https://www.csd3.cam.ac.uk`), provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/T022159/1), and DiRAC funding from the Science and Technology Facilities Council (`https://www.dirac.ac.uk`).

# References


- Raymond M. Smullyan (1978). *What Is the Name of This Book? The Riddle of Dracula and Other Logical Puzzles*. Englewood Cliffs, NJ: Prentice-Hall
- Francis Jeffry Pelletier (1986). "Seventy-Five Problems for Testing Automatic Theorem Provers". In: *Journal of Automated Reasoning* 2.2, pp. 191–216
- Bernhard Beckert and Joachim Posegga (1995). "leanTAP: Lean Tableau-Based Deduction". In: *Journal of Automated Reasoning* 15.3, pp. 339–358
- Jens Otten and Wolfgang Bibel (July 2003). "leanCoP: Lean Connection-Based Theorem Proving". In: *Journal of Symbolic Computation* 36.1–2, pp. 139–161
- Jens Otten (2010). "Restricting Backtracking in Connection Calculi". In: *AI Communications* 23.2–3, pp. 159–182
- Jens Otten (2023). "20 Years of leanCoP — an Overview of the Provers". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*. Vol. 3613. CEUR Workshop Proceedings
- Michael Rawson and Giles Reger (2021). "Eliminating Models During Model Elimination". In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Vol. 12842. Cham: Springer International Publishing, pp. 250–265
- Fredrik Rømming et al. (2023). "Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*. Vol. 3613. CEUR Workshop Proceedings, pp. 107–118
- Armin Biere et al. (2024). "CaDiCaL 2.0". In: *Computer Aided Verification*. Vol. 14681. Lecture Notes in Computer Science. Springer, pp. 133–152
- C. Mangla et al. (2022). "Bayesian Ranking for Strategy Scheduling in Automated Theorem Provers". In: *International Joint Conference on Automated Reasoning (IJCAR)*, pp. 559–577
- Geoff Sutcliffe (2017). "The TPTP Problem Library and Associated Infrastructure: From CNF to TH0, TPTP v6.4.0". In: *Journal of Automated Reasoning* 59.4, pp. 483–502
- University of Cambridge Research Computing Services (2026). *Ice Lake Nodes*. `https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html`. Accessed 13 May 2026

# Any questions?